

CompliSci Platform Modernization

STAR Format: Situation, Task, Action, Result

Situation

CompliSci is a pioneer in regulatory compliance software-as-a-service. Its financial institution customers rely on it to monitor, manage, and identify insider trading and other code of ethics violations. A year-long effort to upgrade the existing technology had failed, and the company engaged me as architect and application manager to rewrite its flagship application from the ground up. When I joined, I found the following challenges:

- The legacy application had eroded the company's competitive edge, resulting in a loss of market share and customers to competitors.
- The core application was not built to support enterprise-level clients, which limited the company to small and mid-sized customers. Every compliance survey revision also required manual code changes, making it slow and costly to meet customer expectations.
- The company lacked the knowledge, skills, and roadmap needed to achieve its goals.

Task

My objective was to modernize the flagship application by rebuilding it from the ground up. The new product had to preserve all existing features and add new capabilities that would let clients, and the company's own teams, grow their market share. Specifically, I needed to stop customer attrition, open the enterprise market to the sales team, and eliminate the need for code changes for routine customer requests. I was also responsible for building the team, process, and roadmap to deliver it while continuing to support the legacy application.

Action

I met with the Product Owners to gather detailed requirements and build a roadmap, studied the legacy application to understand its existing features, and met with the sales team to understand customer pain points and the features needed to win new clients. I then drove innovation in three areas: process, technology, and product.

1. Process

- **Parallel development:** Hired additional engineers to enable parallel development. I divided the team into two scrum teams, one building the new application and one supporting the legacy application, and served as both scrum master and architect for the two teams.
- **Training:** Coached the engineering team through lunch-and-learn sessions with sample code for both front-end and back-end development.
- **Agile adoption:** Trained executive management and the engineering team on the Agile process, and moved the team from waterfall to two-week iterations.

2. Technology

- **Architecture and development:** Laid the foundation of the new architecture while the team was in training. I created design artifacts, built the initial project code, and demoed it to the team.
- **Best practices:** Showed the team how to separate concerns with a layered, service-oriented architecture, and introduced best practices including domain-driven design, design patterns, and service APIs.
- **Engineering tooling:** Modernized source control and revamped the DevOps process with automated build and release management.
- **Security:** Strengthened security by implementing single sign-on.

3. Product

- **Form Builder:** Designed a tool that lets customers create their own compliance forms from standard form components with no code changes. It includes a decision tree that presents follow-up questions based on earlier answers.
- **Company Model Builder:** Replaced the legacy two-role model with a tool where corporate admins define the organization structure, groups, roles, and permissions. Users can belong to multiple groups and roles, so the model scales from a single office to a multi-continent enterprise.
- **Multi-tenancy:** Enabled multiple customers to share a single application instance, where previously each customer ran its own. This gave smaller clients an affordable option without dedicated servers or hosting.
- **Component-based Product Menu:** Let clients choose only the components they need. The professional services team can switch components on or off without code changes.

Launch

After about a year and a half of development, I worked with marketing and sales to demo the beta at the company's annual compliance conference to existing and prospective customers. Interest was strong, so we selected a handful of existing customers as early adopters for a pilot. Over several months we gathered feedback and completed multiple rounds of revisions and bug fixes, and then rolled the application out to the remaining customers.

Result

- Retained 100% of existing customers, and many former customers returned.
- Increased company revenue by 30% in the year following the release.
- Customers gained the ability to make changes themselves without code changes or additional cost.
- The sales team could now market to both small businesses and enterprise-level clients.
- The professional services team could tailor a product to each client, and clients paid only for the services they needed.
- The company was acquired by Vista Equity Partners and publicly announced a 60% increase in its customer base within two years.

Key Learnings

- Modernization succeeds through people and process as much as technology. Investing in training and Agile practices built the capability that the earlier failed effort lacked.
- Starting from customer and sales pain points led to self-service features that solved the root problem instead of speeding up manual work.
- Parallel teams allow innovation without abandoning customers who depend on the legacy system, but the leader must stay closely involved in both.
- A beta demo followed by a pilot with carefully chosen early adopters built customer confidence and produced feedback before the full rollout.